

EXP – 1 Matrix Multiplication

```
#include <stdio.h>
```

```
int main() {
```

```
    int r1, c1, r2, c2, i, j, k;
```

```
    int a[10][10], b[10][10], result[10][10];
```

```
    // Input dimensions for Matrix A
```

```
    printf("Enter rows and columns for first matrix: ");
```

```
    scanf("%d %d", &r1, &c1);
```

```
    // Input dimensions for Matrix B
```

```
    printf("Enter rows and columns for second matrix: ");
```

```
    scanf("%d %d", &r2, &c2);
```

```
    // Matrix multiplication rule check
```

```
    if (c1 != r2) {
```

```
        printf("Error! Column of first matrix must equal row of second matrix.\n");
```

```
        return 1;
```

```
    }
```

```
    // Input elements for Matrix A
```

```
    printf("\nEnter elements of first matrix:\n");
```

```
    for (i = 0; i < r1; ++i) {
```

```
        for (j = 0; j < c1; ++j) {
```

```
            printf("Enter element a%d%d: ", i + 1, j + 1);
```

```
            scanf("%d", &a[i][j]);
```

```
        }
```

```
}
```

```
// Input elements for Matrix B
```

```
printf("\nEnter elements of second matrix:\n");
```

```
for (i = 0; i < r2; ++i) {
```

```
    for (j = 0; j < c2; ++j) {
```

```
        printf("Enter element b%d%d: ", i + 1, j + 1);
```

```
        scanf("%d", &b[i][j]);
```

```
    }
```

```
}
```

```
// Initializing the result matrix with zeros
```

```
for (i = 0; i < r1; ++i) {
```

```
    for (j = 0; j < c2; ++j) {
```

```
        result[i][j] = 0;
```

```
    }
```

```
}
```

```
// Multiplying Matrix A and B
```

```
for (i = 0; i < r1; ++i) {
```

```
    for (j = 0; j < c2; ++j) {
```

```
        for (k = 0; k < c1; ++k) {
```

```
            result[i][j] += a[i][k] * b[k][j];
```

```
        }
```

```
    }
```

```
}
```

```
// Displaying the output
```

```

printf("\nResultant Matrix:\n");

for (i = 0; i < r1; ++i) {
    for (j = 0; j < c2; ++j) {
        printf("%d\t", result[i][j]);
    }
    printf("\n");
}

return 0;
}

```

Output

Clear

▲

Enter rows and columns for first matrix: 2 2
Enter rows and columns for second matrix: 2 2

Enter elements of first matrix:
Enter element a11: 7
Enter element a12: 5
Enter element a21: 5
Enter element a22: 5

Enter elements of second matrix:
Enter element b11: 9
Enter element b12: 2
Enter element b21: 6
Enter element b22: 7

Resultant Matrix:
93 49
75 45

=== Code Execution Successful ===

EXP – 2 Odd or Even number

```
#include <stdio.h>
```

```
int main() {  
    int arr[100], size, i;  
  
    // Get the total number of elements  
    printf("Enter the size of the array: ");  
    scanf("%d", &size);  
  
    // Get the array elements from the user  
    printf("Enter %d numbers:\n", size);  
    for (i = 0; i < size; i++) {  
        printf("Element %d: ", i + 1);  
        scanf("%d", &arr[i]);  
    }  
  
    // Find and display Even numbers  
    printf("\nEven numbers in the given set: ");  
    for (i = 0; i < size; i++) {  
        if (arr[i] % 2 == 0) {  
            printf("%d ", arr[i]);  
        }  
    }  
  
    // Find and display Odd numbers  
    printf("\nOdd numbers in the given set: ");  
    for (i = 0; i < size; i++) {  
        if (arr[i] % 2 != 0) {  
            printf("%d ", arr[i]);  
        }  
    }  
  
    printf("\n");  
    return 0;  
}
```

}

```
Output Clear
Enter the size of the array: 5
Enter 5 numbers:
Element 1: 5
Element 2: +
Element 3: 5
Element 4: 52
Element 5: 52

Even numbers in the given set: 0 52 52
Odd numbers in the given set: 5 5

=== Code Execution Successful ===
```

EXP – 3 Factorial of a given number without using Recursion

```
#include <stdio.h>

int main() {
    int num, i;

    unsigned long long factorial = 1; // Used to handle large numbers

    printf("Enter an integer: ");

    scanf("%d", &num);

    // Factorial is not defined for negative numbers
    if (num < 0) {
        printf("Error! Factorial of a negative number doesn't exist.\n");
    } else {
        // Loop to multiply numbers from 1 up to num
        for (i = 1; i <= num; ++i) {
            factorial *= i; // Same as factorial = factorial * i;
        }
    }
}
```

```

    printf("Factorial of %d = %llu\n", num, factorial);
}

return 0;

```

Output

```

Enter an integer: 5
Factorial of 5 = 120

```

```

=== Code Execution Successful ===

```

EXP – 4 Fibonacci series without using Recursion

```

#include <stdio.h>

int main() {
    int n, i;

    unsigned long long t1 = 0, t2 = 1, nextTerm;

    printf("Enter the number of terms: ");

    scanf("%d", &n);

    if (n <= 0) {
        printf("Please enter a positive integer greater than 0.\n");
    } else {
        printf("Fibonacci Series: ");

        for (i = 1; i <= n; ++i) {
            printf("%llu ", t1);

            // Calculate the next term and shift variables
            nextTerm = t1 + t2;

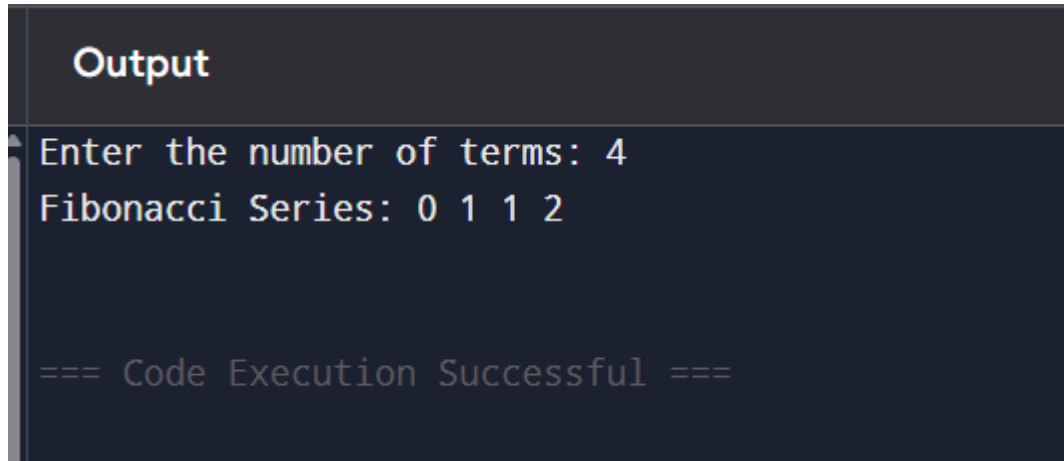
            t1 = t2;
            t2 = nextTerm;
        }
    }
}

```

```

    }
    printf("\n");
}
return 0;
}

```



The screenshot shows a dark-themed terminal window with the title "Output". The text inside the terminal reads: "Enter the number of terms: 4", "Fibonacci Series: 0 1 1 2", and "=== Code Execution Successful ===".

EXP – 5 Factorial of a given number using Recursion

```

#include <stdio.h>

// Function declaration
unsigned long long multiplyNumbers(int n);

int main() {
    int num;

    printf("Enter a positive integer: ");
    scanf("%d", &num);

    if (num < 0) {
        printf("Error! Factorial of a negative number doesn't exist.\n");
    } else {
        printf("Factorial of %d = %llu\n", num, multiplyNumbers(num));
    }

    return 0;
}

```

```
// Recursive function definition
unsigned long long multiplyNumbers(int n) {
    if (n >= 1) {
        return n * multiplyNumbers(n - 1); // Function calls itself
    } else {
        return 1; // Base case: 0! = 1
    }
}
```

Output

```
Enter a positive integer: 4
Factorial of 4 = 24
```

```
=== Code Execution Successful ===
```